Introductory Notes

This tutorial (in five parts) is optimized for poets/artists; it encourages experimentation, play, rebellion, and meaningful content. It also prioritizes quickness. While we will not learn cheats of any kind, we will skip many things covered by traditional tutorials that are not conducive to immediate interesting results. Also, though there are often many ways to achieve the same thing in code, we'll move on after learning one or two ways.
Notes on usage:

- Every tutorial includes a link to a webpage that could be made using that tutorial, along with its code. Hopefully the example gets you excited about what's the come; its code may be helpful to compare against yours, if you get stuck.
- But! Don't copy the example code and use it as a base; it will be confusing and dampen your own creative agency. In general, avoid copy-and-pasting throughout the tutorial unless I say otherwise explicitly.
- When you type out the code snippets, be sure to copy all punctuation exactly, or your code may not work.
- Brackets are the only exception: **brackets in code snippets indicate texts that will vary for everyone** (ie, the textual content of your page), and **<u>the brackets themselves should not be included.</u>**
- Every step includes a change you should make in your code or an action you should take. When there's more text for a certain step, that just means I've included more explanation.
- Don't be afraid of code!

Tutorial A: Intro to HTML and CSS

1. Here is an example of what your webpage could look like at the end of Tutorial A. The code that makes it is at the bottom of this tutorial. I'll include an example like this for each tutorial, so you know what you're getting into. Don't copy-and-paste this code and use it as a base! You will learn better writing the code from scratch; it's just there if you get lost and want to check your code side-by-side against mine.

2. First, make the folder that will house your website and only your website, naming it something that follows these two **naming rules:** 1) Keep everything lowercase and 2) Avoid spaces and punctuation (other than the dash and underscore). Throughout this tutorial, you'll often have to name something, and I encourage you to have fun and/or wax poetic with your names. Think of them as street names: it doesn't matter what a street is called, as long as everyone calls it the same thing. As your website gets more complicated, you'll refer to files, folders, and elements in several different moments. As long as you keep their names consistent and follow the naming rules, you can call them anything you want.

3. Now that you have your folder, download Visual Studio Code (VSC), a lightweight and free text editor. There are many good text editors, but Visual Studio Code is one of the most popular. Click the appropriate download for your computer, and then open the app.

4. Close out the "Get Started" tab. Then, click File->New text file. Save that file in your website folder as "index.html"

5. Go to View->Word Wrap, so you can always see all your code and/or text.

6. Write the following code in your index.html file (remember to always avoid copy and pasting unless otherwise noted):

```
<!doctype html>
  <html>
    <head>
      <style>
      </style>
    </head>
    <body>
    [Any text you wish. Found text is welcome!]
    </body>
  </html>
```

7. Don’t worry if the colors used by your version of VSC are different from those above! The colors are meant to help you find errors, but they may not be the same as mine, depending on the VSC version. As always throughout this tutorial, brackets indicate texts that you may choose. The brackets themselves don’t need to be included! I encourage writing strange things directly into the code, or using found text. Anything is welcome, from a poem, to a Wikipedia entry on an obscure historical figure or a list of botanical descriptions of various roses. Just make a note of your source (links and/or title/author will do) somewhere; I’ll have you add them into the code later on. Lorem Ipsum is strongly discouraged!

8. Congrats! You've made your first webpage. It’s written in **HTML (Hypertext Markup Language)**. Save it, right-click on your index.html file in Finder, and select "Open with [your favorite browser]." Double-clicking on the file will also probably work. You should see whatever you wrote in step 3

displayed in your browser—if you don't, check to make sure you typed everything perfectly. You'll want to keep this page open in your browser as you work. Whenever you want to check that you've done something right, you should 1) save all your files (command + S or control + S) and 2) refresh the page in your browser (command + R or control + R). You'll be doing this constantly, so I recommend getting used to those keyboard shortcuts! This webpage is now **running locally**, which means it can only be viewed on your specific computer; we'll talk about hosting on the WWW later on. The code in step 3 is the skeleton of most standard webpages. Here's what it all means:

- The <html> and <!doctype html> tags alert any program or person that the document is written in HTML. The </html> tag, at the very bottom of the code, indicates the end of the HTML area. Any tag with a "/" is a **closing tag.**

- Everything between the <body> tag and the </body> tag (ie, everything **"in the body"**) is the material of your webpage. I like to think of this as my toolkit: it's where I put all the images, text, etc. that I'm going to use on my page.

- Everything between the <head> tag and the </head> tag (ie, everything **"in the head"**) molds what's in the body or describes what's in the body (for external manifestations of the webpage like its tab or the information it would show on a Google search).

- The molding (or styling) aspect of the head happens between the <style> tag and the </style> tag. That's actually a separate coding language, called **CSS (Cascading Style Sheets)**, and when something needs to be between those two tags, I'll say it needs to be **"in the CSS"**.

- NOTE: Spacing does not matter here, as far as the browser is concerned; if all this code were on one line, it would still run. The indents and line-breaks I include in this tutorial are for humans: they help you remain organized, will resemble what you'll see in online Q&A forums, and will make it easier for peers to read your code.

9. Now, to make that text a bit more visually fun, let's write your first bit of CSS. First, we need to define our text as an **element** with an **id**, so we can assign specific CSS to that specific part of the body. There are many different kinds of text elements, but we're going to make this text a **div**, which is a very common text element type. First, add " before your text, choosing an id (anything you wish!) that follows the naming rules in step 2, and " " after your text, to close it out -- like so:

```
      <!doctype html>
      <html>
        <head>
          <style>
          </style>
        </head>
        <body>
          <div id="[your-div-id]">[Any text you wish. Found text
welcome!]</div>
        </body>
      </html>
```

10. Then, in the CSS (again, that's between the <style> tag and the closing </style> tag), add the following:

```
#[your-div-id] {
        position: fixed;
        color:red;
        background-color:#99004d;
        text-shadow: 1px 3px 1px white;
        border: 6px solid black;
        width: 300px;
        left: 300px;
        top: 100px;
```

```
        font-size: 90px;
        font-family: Helvetica;
        text-align: right;
        }
```

11. **After saving your index.html file,** refresh your page in your browser to see what's happened. Before I tell you what all that means, play: change everything between a colon and a semicolon (that is, every value in every declaration). Don't be afraid to be extreme! Broken code can be beautiful.

    - NOTE: Everything between the curly brackets in CSS styles the **element** (or section of the body) that's named right before the curly brackets—in this case, everything in the div with the id you've specified. The texts that directly precede the colons are called **properties**, and the texts between the colons and the semi-colons are called **values**. Together, a property and its value make a **declaration**.

12. Before we go any further, let's learn how to **comment out,** or deactivate, certain bits of code – so you can write notes to yourself or disable a design element you're not sure about. You can think of commented out text as human-only: for the browser, it will be as good as deleted. Each language we work with in this tutorial has different punctuation for commenting out, so I recommend using the keyboard shortcut: command + / or control + /. Now, if you used found text, paste the link and/or citation at the very bottom of your code, and comment it out. Otherwise, try commenting out a note to yourself. Below, what's commented out is in green; notice that the punctuation is different in CSS and HTML:

```
<html>
   <head>
       <style>
       #inmyfinger {
           color:red;
           /* color: blue; */
       }
       </style>
```

```
    </head>
    <body>
        <div id="inmyfinger">I want a whole wheel</div>
        <!-- <div>I want half a wheel</div> -->
    </body>
</html>
```

13. Now, let's keep playing with CSS – but with a bit more guidance. The first thing you need to do to an element in CSS is define its “position,” or the way it is positioned relative to the other elements — otherwise, the rest of your CSS may not function correctly. In this tutorial, we're going to use two position values: "fixed" and "absolute." Fixed elements are not scrollable, whereas "absolute" elements are. To see how this works, add a lot of text to your div and/or make the font size very large, so that the div goes past the bottom of the page. Save and refresh; you’ll see you can’t scroll. Now, change “fixed” to “absolute”, save, and refresh. Now you can scroll!

14. "Color" defines the color of the text, and "background-color" defines the color of the background. Many basic color names will work as color values, like “red” in step 10. However, if you want to be more specific, you can choose a color here and copy the Hex value (which will always be a hashtag followed by six digits) – as I did for the “background-color”. Make sure you’ve changed both the background color and the text color of your div.

15. The first and second values of "text-shadow" describe where the shadow goes relative to the text. Make sure to change both these values, making at least one of them negative. The third value describes the degree to which that shadow is blurred; the last value is the color of the shadow. As above, this can be defined by a basic color name or by a Hex value. Make sure you’ve changed these too.

16. The first value of "border" defines its width; the second value defines its style; and the third value defines its color (once again, a basic color name or a Hex value). The different border styles you can try are: dotted, dashed,

solid, and double. Try changing "border" to "border-bottom" and see what happens. There's also, of course, "border-right," "border-left," and "border-top".

17. "Width" defines the width of the element; make sure you've also changed that! You can also add a declaration with the property "height" and the value of your choice. Play around with defining both of these.

18. "Top" and "left" define the distance of the element from the top and left of the window respectively. In some cases, it might be more appropriate to define the distance from the "bottom" than from the "top," or more appropriate to define the distance from the "right" than from the "left." Defining both "top" and "bottom" OR both "left" and "right" will lead to one of those declarations being ignored. Try changing "top" to "bottom" and try changing "left" to "right" – just to see what happens. Play around till your div is where you'd like it.

19. "Font-size" is font size. Switch it up!

20. "Font-family" defines the font style. Here is a list of web-safe fonts that you can just type in. If you want more options, go to Google Fonts, click the plus sign next to the font you like, go to the tab at the bottom, and follow the instructions.

21. Now, change the alignment of your text via the "text-align" property. The values this property accepts are "left," "right," "center," and "justify."

22. Alright! Let's move on from the text and add an image. Choose any image (remembering that gifs are technically images – in my example, I used gifs with transparent backgrounds, found online) and move it to the same folder that houses your index.html file. Feel free to use an image you took/created, or something found online. Just make note of the link/source, along with the link/source(s) of any found text you've used, and add below your code, commented out.

23. Make sure your image name follows the naming rules and that it has an **extension** (or the part after the period) —for example, bigbutter.jpg or we-went-while.gif. If your image doesn't have an extension in the file name

already, check what type it is and add the appropriate extension to the file name.

24. In the body, before your div, write the following line of code:

```
<img id="[your-image-id]" src="[your-image-name]">
```

25. In place of [your-image-name], write the file name of your image, including the extension; in place of [your-image-id], make up a name for your image, again following the naming rules. The id for your image can be totally different from the file name of your image! But it can also be the same, if that's easier. Notice that there's no closing tag necessary for images; it's the only exception to that rule we'll encounter in this tutorial. When you refresh your page in your browser, your image should show up on the top of your page.

26. Let's mobilize some CSS for your image. Add the following code to your CSS; it doesn't matter if this is before or after the code you wrote in step 10:

```
img {
  position: fixed;
  top: 0px;
  right: 0px;
  width: 1200px;
  opacity: .6;
  border-radius: 15px 50px 30px 5px;
}
```

27. This code should make your image appear differently. Now, as with the CSS for your text, change and play with everything till your code looks very different from the code I've offered above.

28. Some suggestions for experimentation: Check what happens if you define both the width and the height of your image; try making either the top or right values negative; play with "opacity," which takes values 0-1; and try

subbing % for px in the border-radius values, which rounds the corners of your image.

29. Notice that I didn't use the id to call up this image, but instead "img." Try replacing "img" with "#[your-image-id]" – then, try replacing "#[your-div-id]" with "div." In both cases, nothing on your page should change. When "div" alone comes before curly brackets, the CSS between those curly brackets refers to all divs on your page; similarly, the CSS in the curly brackets after "img" refers to all images on your page. Right now, because you have only one image and only one div on your page, it doesn't matter whether you refer that specific element by its id or to all elements of that type.

30. To see this in action, add a second image to your page. Follow the instructions exactly as you did for your first image, but give your second image a different id. Then, give some css to all images (using "img" before the curly brackets) and some css to each of your two images (using "#[your-image-id]" and "#[your-second-image-id]" before the curly brackets).

31. Feel free to add more images! You can even add the same image twice, give it a different id, and assign different CSS to it.

32. You can give CSS to something even bigger/broader: the body itself. In your CSS, write "body" before a set of curly brackets – and give that body (in other words, the entire page) a background-color. Some people like to define other things for the entire body, like a font or color; what you put here will apply to everything on your page, though, so I often keep it minimal. Here's the code:

```
body {
        background-color: black;
       }
```

33. Let's put in a third type of element: video. Place a video (as always, this can be found online or original; just make note of the link/citation and comment it out in your code) in the same folder as your index.html file, like

you did with your images. The video tag currently supports .mp4, .ogg, and .webm on the most popular browsers—so, if your video is not in one of those formats, use an online converter to make it so. Sometimes it also works to just change the extension of the file itself. Then, type this code into your body (reminder: always try to type rather than copy-and-paste):

```
<video loop controls id="[your-video-id]" src="[your-video-name]"
type="video/[your-video-format]"></video>
```

If your video is .mp4, “[your-video-format]” should be "mp4"; etc. As with your image, “[your-video-name]” should include the extension. And, as always, your id and the name of your file can be anything you want, as long as they follow the naming rules. You don't need "loop" if you don't want your video to loop, but do include "controls"; it’s the easiest way to make a play button.

34. You can probably guess how to add CSS to your video – try to do it on your own before reading further.

35. If you were stuck, here’s how: add “video” or “#[your-video-id]” before curly brackets. Most of the properties that apply to images also apply to videos. Place your video where you’d like it to be on your page. At minimum, you'll want to define your video's position; width OR height (you can have both but need at least one); top OR bottom (one or the other); and left OR right (one or the other). Here’s an example of what the CSS for your video might look like:

```
    video {
        position: fixed;
        width: 10vw;
        top: 9vh;
        right: 60vw;
        border-radius: 0px 100px 10px 0px;
        z-index: 10;
```

```
  }
```

36. You'll notice a new property here: **z-index**. As a general rule, elements layer on top of one another in the order they appear in your body, so that the last element in your body is the top layer. You won't be able to press play on your video if it's covered by something else (like an image with less than full opacity). We'll discuss this further next week, but for now, if you're having trouble pressing play on your video, feel free to add "z-index: 10;" into its CSS.

37. I also snuck in a new kind of measurement in that code snippet: **vh/vw** (or **viewport height** and **viewport width**, respectively). There are many ways to define measurements in code, but we're going to stick to px, vh, and vw. While what's measured in pixels (px) will remain the same size regardless of how large your screen is, what's measured in vh/vw will change relative to the height and width of your screen, respectively – where 100vw is the full width of your screen. Using vh/vw will help ensure that your site looks more or less the same on differently-sized browsers and/or devices; in other words, it's your first step toward making your site **responsive** (more soon). Change some of your measurements to vw or vh, and then resize your screen to see what happens. And don't worry about mixing vw, vh, and px for one element or even one property (if that property takes various numerical values); with very few exceptions, this will work just fine.

38. Congrats - you've finished the first tutorial. Feel free to play more, adding more images, texts, or videos, and giving each new element its own CSS using ids. In the next tutorial, we'll learn more tricks with CSS, including animation, and start making your webpage into a **website** via links!

Here's the code that made the example:

```
<!doctype html>
   <html>
     <head>
       <style>
```

```
body {
     background-color: blue;
     }

#willothe {
     position: absolute;
     color:blue;
     background-color:#004D39;
     text-shadow: 18px -300px 12px black;
     border: 60px dotted black;
     width: 9vw;
     height: 2700px;
     right: 600px;
     top: 300px;
     font-size: 900px;
     font-family: Helvetica;
     text-align: center;
     }

 #goingto {
     position: fixed;
     font-size: 12px;
     top:30vh;
     left: 30vw;
     }

 .runofthe {
 color:red;
      }

 img {
    border-left: .15vw solid black;
      }

 #swayfish {
     position: absolute;
     top: 90vh;
```

```
        right: 0px;
        width: 30vw;
        height: 90vw;
        z-index: 10;
        opacity: .6;
        }

    #swayfishtwo {
        position: absolute;
        top: 600vh;
        left: -45vw;
        width: 180vw;
        height: 45vw;
        z-index: 10;
        opacity: .6;
        }

    #swayfishthree {
        position: absolute;
        top: 450vh;
        left: 45vw;
        width: 6vw;
        height: 3vw;
        z-index: 10;
        opacity: .6;
        }

    #blackfish {
        position: fixed;
        top: 30vh;
        z-index:11;
        right:-21vw;
        width: 60vw;
        height: 60vw;
        opacity: .74;
        }

    #blackfishtwo {
```

```
            position: fixed;
            top: -10vh;
            z-index:11;
            right:-30vw;
            width: 90vw;
            height: 45vw;
            opacity: .6;
            }

        #blackfishthree {
            position: fixed;
            top: -36vh;
            z-index:11;
            left:-30vw;
            width: 120vw;
            height: 45vw;
            opacity: .6;
            }

        #goldie {
            position: fixed;
            bottom: 69vh;
            z-index:11;
            right:3vw;
            width: 45vw;
            height: 18vw;
            opacity: .6;
            }

        video {
            position: absolute;
            width: 30vw;
            border-radius: 30% 60% 15% 10%;
            top: 15vh;
            left: 6vw;
            }

    </style>
```

```
        </head>
        <body>
        <img id="swayfish" src="swayfish.gif">
        <img id="swayfishtwo" src="swayfish.gif">
        <img id="swayfishthree" src="swayfish.gif">
        <img id="blackfish" src="blackfish.gif">
        <img id="blackfishtwo" src="blackfish.gif">
        <img id="blackfishthree" src="blackfish.gif">
        <img id="goldie" src="goldfish.gif">
        <div id="willothe">I will I wit to whit I will of the in the
great big tree</div>
        <div id="goingto">gonna</div>
        <div class="runofthe" id="goingto">gonna</div>
        <div class="runofthe"
id="goingtobedinthestickysummer">going</div>
        <div class="runofthe" id="goingtobed">full</div>
        <video loop controls id="sam" src="weed4.mp4"
type="video/mp4"></video>
        </body>
      </html>




    <!-- IMAGE SOURCES

https://www.google.com/search?q=gif+of+fish+transparent+&tbm=isch&ved
=2ahUKEwjf1OiZi8D8AhUQEFkFHTXnBtwQ2-cCegQIABAA&oq=gif+of+fish+transpa
rent+&gs_lcp=CgNpbWcQAzIGCAAQCBAeOgQIIxAnUPcLWNwlYMsnaAJwAHgAgAFIiAHO
B5IBAjE1mAEAoAEBqgELZ3dzLXdpei1pbWfAAQE&sclient=img&ei=-Pe-Y9_4GJCg5N
oPtc6b4A0&bih=738&biw=1281&hl=en#imgrc=NdicPCUD60VmoM
    https://opening.download/spring-2021.html  -->
```

Tutorial B: All the Fun with CSS

1. Here is the example of what your webpage could look like at the end of Tutorial B. The code that makes this page, as always, is at the end of this tutorial. Don't copy, but do reference if you get stuck!

2. This tutorial builds on the last one – so open up the index.html page you already began, in both VSC and your browser. You're welcome/encouraged to change/delete/add any content as you go, but you'll need its basic infrastructure.

3. First, let's personalize the tab of your webpage and get it looking fancy. Right now, it probably just says "index.html." I mentioned in the last tutorial that what goes in the head not only molds what's in the body (via CSS) but also describes what's in the body for external manifestations of the webpage. One example of this is the tab of your webpage. Add this code anywhere in the head (but not in the CSS) of your webpage:

```
<title>[whatever you want on your tab]</title>
```

4. Save and refresh the page. On the tab for your page, you should see what you wrote.

5. Choose the image you want to be your **shortcut icon** or **favicon –** or the image next to your title on the tab (if you find it online, as always, copy-and-paste some information about its source in your code, and comment it out.) Keep in mind that the image will be made 16px by 16px—so a very wide image will be stretched vertically, a very tall image will be stretched horizontally, and large images will be shrunk. I

recommend using a photo editor to make your image strangely-shaped, or looking for something online with a transparent background.

6. If the image is not already in .png, .gif, or .ico format, use an online converter to make it so.

7. Move your image to the same folder that houses your index.html file, making sure to include the correct extension (.png, .gif, or .ico). Then, insert this code in the head (but not in the CSS) of your .html file:

```
  <link rel="shortcut icon"
href="[your-favicon-file-name-with-extension]">
```

8. Let's do a slightly less-fun thing while we're here (in the head, but not in CSS). Add the following code:

```
<meta charset="UTF-8">
```

This probably won't change anything on your page, especially since you're running it locally right now. In general, however, it ensures that the browser can read and render all the characters you type into your body – especially important with punctuation and languages other than English.

9. Now, let's put some links into your webpage. You think you know what a **link** is, but let's make sure: it's the the element on a webpage (text, image, or video) that, when clicked, opens a different webpage. People sometimes refer to a **url** (Uniform Resource Locators, or the specific address of a webpage) as a "link," and that's fine – but in this tutorial (and technically), a link is that which leads to a url. To make an element into a link, add "<a href=[your-link-url]>" before it, and "</a>" after it. Make sure your link includes the full path (https, etc.); "google.com" will not work. Here's an example of a few different elements as links:

```
      <a href="[your-link-url]"><img
src="[your-image-file-name]"></a>

      <a href="[your-link-url]"><video loop controls
src="[your-video-file-name]"></video></a>

      <a href="[your-link-url]"><div>[Any text]</div></a>

      <div>[Any <a href="[your-link-url]">other</a> blue bell]</div>
```

10. Notice that you can make only part of your text into a link, as in the final line of code above. Experiment with making at least two different elements into links, thinking about using links to draw surprising/intense connections.

11. Just like play buttons in the previous tutorial, a link won't work if it's covered up – even if what's covering it has less than full opacity. So, if you're having trouble with your link, try giving it a large z-index, as we did with the videos.

12. Now, let's play with **z-index** a bit more. You may remember something about the z axis from high school, when discussing graphs: x axis is width, y axis is height, and z axis is depth. The z-index property reconfigures the layers of your webpage, determining which goes on top of which. Because we're not AFK (away from keyboard), z-index will not affect the size of the element or give the appearance of depth; it will just determine whether or not it covers up other elements. All elements without a defined z-index are on layer 0. Try this out by moving an element behind all other elements by giving it a z-index of any negative number; then, move another element in front of all other elements by giving it z-index of any positive number.

13. Let's look at an example to get this straight:

```
  #[the-id-of-element-a] {
  z-index:1;
}

#[the-id-of-element-b] {
```

```
  z-index: -1;
}

#[the-id-of-element-c] {
  z-index: 10;
}
```

Assume there is also an element d with no defined z-index. Element b will be behind everything; element d will be in front of b but behind everything else; element a will be in front of element b and d but behind c; and element c will be in front of everything. Z-index hilariously accepts values from –2147483648 to +2147483647, but people tend to stick to low numbers to avoid confusion. If element c had any z-index between 2 and 2147483647, these elements would have the exact same appearance on the page. Play around with adding z-index to many or all of your elements, till everything on your page is layered exactly how you want; of course, feel free to add new elements and play with those too.

14. Back to your link: you'll notice that textual links are automatically formatted with underlines and blue color (for unvisited links) or purple color (for visited links). You may not like this! To give links CSS, we use **pseudo-class selectors**. First, make sure one of your links is a textual link, and give it an id (between "<a" and "href>"), like so:

```
  <div>[Any <a id="[the-id-of-your-link]"
href="[your-link-url]">other</a> blue bell]</div>
```

15. Then, add this into your CSS:

```
    #[the-id-of-your-link]:link {
      color: black;
      text-decoration: none;
    }
```

```
#[the-id-of-your-link]:visited {
  color: green;
  text-decoration: none;
}

#[the-id-of-your-link]:hover {
  color: red;
  font-style: italic;
}

#[the-id-of-your-link]:active {
  color: blue;
}
```

16. Note that there are no spaces on either side of the colon, here: this is one situation where the code will break if you add spaces. Now, save your code and refresh your page. Assuming you've visited your link, it should show up as green and without any underline (it would be black if you'd never visited it before.) When you hover over it, it should change to red. When you click on it, it should flash blue before taking you to the page – though this may not happen on all browsers. Now, experiment by adding more CSS within those brackets, remembering that anything goes! What if hovering over a link changes the text-shadow, line-height, letter-spacing, width, or its location on the page (via top/bottom/left/right)? If any of this breaks the code (or causes flickering), think about why.

17. The hover pseudo-class selector actually works for all elements, regardless of whether or not they are links. Use this to make an image change height when you hover over it.

18. Now, let's slow that change down, and make an animation. To do this, we're going to use the **transition** property. Add "transition: height 3s ease-in;" into the CSS of the image. A very simple example looks like this:

```
#goldie {
      position: fixed;
      top: 6vh;
```

```
            z-index:11;
            right:3vw;
            height: 18vh;
            transition: height 3s ease-in;
            }
      #goldie:hover {
          height: 60vh;
           }
```

19. In the above example, the transition declaration translates to: make the transition of the height upon hover take 3 seconds, and make the rate of that transition slower at the beginning. If you don't include that third value, the rate of change will be linear. Try replacing "ease-in" with "ease," "ease-out," or "ease-in-out."

20. You can slow down the transition of many other properties on all different kinds of elements! There are exceptions – for example, the browser won't know how to slowly transition from one font-family to another – but experiment, see how far you can push the code.

21. Now that you know how to have fun with your CSS, let's learn some more organizational skills. Make sure you have at least three divs. This may require you to find more text – as always, found texts and nonsense are welcome (just comment out information about your sources.)

22. Give all your divs an id – have fun with these, as long as you follow the naming rules.

23. Now, let's use a **class** to give the same properties to multiple divs – since an id only allows us to give properties to one element. You give an element a class in the same way you give an element an id, replacing "id" with "class." It's common and useful for an element to have both a class and an id. Classes, like ids, can be anything you want, as long as you follow the naming rules. Give at least two of your divs the same class. Your divs might look something like this:

```
<div class="runofthe" id="goingto">gonna</div>
```

```
<div id="goingtobedinthestickysummer" class="runofthe">going</div>
<div class="runofthe" id="goingtobed">full</div>
```

24. As you can see in line 2 of the above code, it does not matter whether you state a class or id first, in the opening tag. Now, let's give some CSS to that whole class of divs, starting with one simple declaration:

```
.runofthe {
      color:red;
    }
```

25. Give some CSS to that whole group of divs (using class), and specify different CSS you want for individual divs (using id). Do this with different properties! If you have conflicting declarations (ie, if you've defined a black color using the class of a div, and a red color using the id of the div), one of those will be ignored. Here's a quick review of how to reference elements in CSS:

    - "." translates to, "I'm talking about the elements with the class of:"
    - "#" translates to, "I'm talking about the element with the id of:"
    - no punctuation (ie, div, img, a) translates to, "I'm talking about all elements of this type"

26. Now, let's practice/play by combining what you've learned, without coding examples from me. Classes and ids work the same way for images – so, give a few of your images the same class.

27. Then, make something happen to all images in that class when you hover on them (for example, drastically change their opacity).

28. Now, make it so that the transition happens slowly.

29. Let's go back to your text. You may notice, if you have line breaks (or paragraph breaks) in the text you entered into the code, that those line breaks have been collapsed. One way to add line breaks is to type "<br>" right into your text where you want the break. Do this once, save, and refresh to see it at work; I'll show you another way in the next step.

30. Now, make sure that there's at least one line break in one of your divs, in the code – feel free to just press "enter" arbitrarily, if you're working with prose. Add "white-space: pre;" as a declaration for your entire body in your CSS, so that the CSS for your body looks like so:

```
body {
 background-color: [whichever-background-color-you-chose;
 white-space: pre;
}
```

31. Save and refresh; you'll see that your line break(s) have been preserved! I have a weird fondness for using "<br>" to create my line breaks – but either way works.

32. Now, let's learn how to give CSS to certain words within your divs. Though there are many different textual elements, their functionality doesn't vary much—so we're only going to deal with spans and divs in this tutorial. While a div (which is short for "division") is a *block* element, **spans** (which come from the homonymous verb) are *inline* elements. What this means:

    - A div causes a line break before/after it; a span does not.

    - A div can act as a container for many different kinds of content—for example, it can include both text and image—whereas a span might get glitchy if asked to do so.

    - Spans (often) go inside divs. They are especially useful if you want to add formatting to one word or one phrase within a larger text (which you're formatting as a div).

33. You make a span much in the same way that you make a div: add "" before the text and "" after the text. Here's an example of a span being added into a div:

```
<div id="cuernocueva">Once upon a midnight <span id="dear">dreary as
I pondered</span> weak and weary</div>
```

    Now, I can apply CSS to "dreary as I pondered" and nothing else in the div (using #dear). Because the span is inside the div, it will take the CSS of the div as default; however, the CSS for the span will override the CSS for the broader div. Make at least two spans within your divs.

34. Now, we're going to give your spans some CSS. If there were only one span on your page, you could type "span" before your curly brackets in your CSS; however, since you have at least two spans, use ids.

35. You already have the tools to make a page that is unsettling, surprising, and/or meaningful – go wild and play. Add more texts, images, or videos, and practice turning them into links, giving them CSS, or adding animation on hover. Experiment with everything: try very large numbers and very small numbers, try adding properties to elements you didn’t think would take them, etc. Remember that, if you break something, there’s always command/control + Z. In addition to the properties we already learned, try out these: font-weight, letter-spacing, line-height, text-transform, word-spacing, and text-decoration. Here is a big list of declarations you might want to toy with (it also tells you specifically what values are allowed for what properties.)

36. One last bit of fun: CSS can also add filters to the images themselves. While you probably want to do most image editing using other software, you might want to use these filters with the hover pseudo-class. This site shows many different ways you can edit your image using CSS.

Here’s the code that made the example (I’ve redacted the huge amount of found text in the background, where noted):

```
<!doctype html>
<html>
 <head>
   <title>Pander-Toe</title>
   <link rel="shortcut icon" href="fish.png">
   <meta charset="UTF-8"/>
       <style>
        body {
             background-color: blue;
             }

/* HERE'S CSS FOR THE IMAGES AND VIDEOS */

           img {
               border-left: .15vw solid black;
               }

           #swayfish {
               position: absolute;
               top: 90vh;
               right: 0px;
               width: 30vw;
               height: 90vw;
               z-index: 10;
               opacity: .6;
               }

           #swayfishtwo {
               position: absolute;
               top: 600vh;
               left: -45vw;
               width: 180vw;
               height: 45vw;
               z-index: 10;
               opacity: .6;
               }

           #swayfishthree {
```

```
        position: absolute;
        top: 450vh;
        left: 45vw;
        width: 6vw;
        height: 3vw;
        z-index: 10;
        opacity: .6;
        }

    #blackfish {
        position: fixed;
        top: 30vh;
        z-index:11;
        right:-21vw;
        width: 60vw;
        height: 60vw;
        opacity: .74;
        }

    #blackfishtwo {
        position: fixed;
        top: -10vh;
        z-index:11;
        right:-30vw;
        width: 90vw;
        height: 45vw;
        opacity: .6;
        }


    #blackfishthree {
        position: fixed;
        top: -36vh;
        z-index:11;
        left:-30vw;
        width: 120vw;
        height: 45vw;
        opacity: .6;
```

```
            }

        #goldie {
            position: fixed;
            top: 6vh;
            z-index:11;
            right:3vw;
            width: 45vw;
            height: 18vh;
            opacity: .6;
            transition: height 3s ease-in;
            }

        #goldie:hover {
            opacity:1;
            height: 30vh;
            }

        .fish{
            transition: background-color 3s;
        }

        .fish:hover {
            filter: invert(100%);
            background-color: orange;
        }

        video {
            position: absolute;
            width: 30vw;
            border-radius: 30% 60% 15% 10%;
            top: 15vh;
            left: 6vw;
            }

/* HERE'S CSS FOR THE TEXTS */
        #willothe {
            position: absolute;
```

```
            color:blue;
            background-color:#004D39;
            text-shadow: 18px -300px 12px black;
            border: 60px dotted black;
            width: 9vw;
            height: 2700px;
            right: 600px;
            top: 300px;
            font-size: 75vh;
            font-family: Helvetica;
            text-align: center;
            line-height: 75vh;
            }

        #goingto {
            position: fixed;
            font-size: 12vw;
            top:30vh;
            left: 50vw;
            z-index:100;
            line-height: 9vw;
            font-style: italic;
            }

        .runofthe {
            color:red;
            font-family: Garamond;
             }

        #goingtobedinthestickysummer {
            position: fixed;
            background-color: blue;
            width: 30vw;
            height: 30vh;
            text-align: right;
            transition: border-radius 2s ease-out;
            /* opacity: .6;  */
            }
```

```
#goingtobedinthestickysummer:link {
    text-decoration: none;
    z-index:100;
    }

#goingtobedinthestickysummer:visited {
    text-decoration: none;
    z-index:100;
    }

#goingtobedinthestickysummer:hover{
    font-size: 12vw;
    font-style: italic;
    border-radius: 50%;
    letter-spacing: 2vw;
    }

#goingtobedinthestickysummer:active {
    color: blue;
    }

#grow:hover {
    font-family: Garamond;
    letter-spacing: -9vw;
    border-right: 3vw red dashed;
    width: 100vw;
    /* font-style:normal;  */
    }

#grow {
    transition: letter-spacing 6s;
    }

#wont {
    transition: border-right 3s;
    }
```

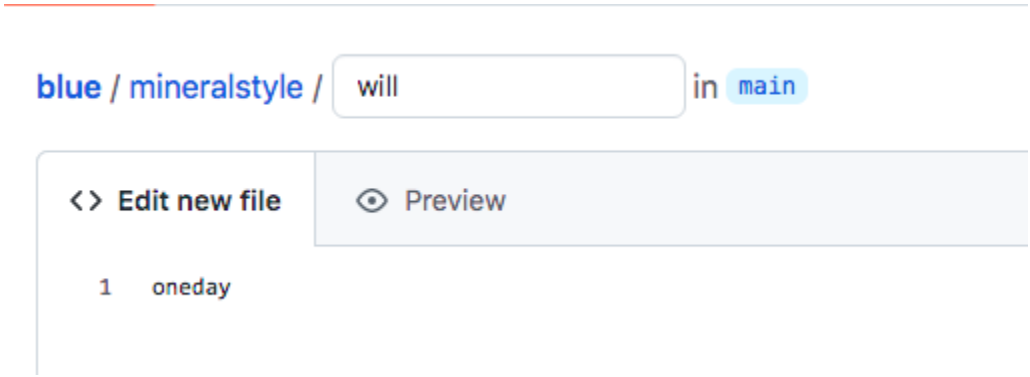
```
            #wont:hover {
                font-family: Helvetica;
                border-right: 3vw red dashed;
                }

            .ow {
                text-shadow: 1vh 1vh 3vh orange;
                }
        </style>
    </head>
    <body>
    <img class="fish" id="swayfish" src="swayfish.gif">
    <img class="fish" id="swayfishtwo" src="swayfish.gif">
    <a
href="https://www.google.com/search?q=wishbone&tbm=isch&chips=q:wishb
one,g_1:bone:e-ZwXTt7yFE%3D&hl=en&sa=X&ved=2ahUKEwicutKsudH8AhVdrHIEH
bcOCH4Q4lYoAHoECAEQJg&biw=1360&bih=786"><img id="swayfishthree"
src="swayfish.gif"></a>
    <img class="fish" id="blackfish" src="blackfish.gif">
<img id="blackfishtwo" src="blackfish.gif">
    <img class="fish" id="blackfishthree" src="blackfish.gif">
    <img id="goldie" src="goldfish.gif">
    <video loop controls id="sam" src="weed4.mp4"
type="video/mp4"></video>


    <div id="willothe">I will</a> I wit to whit I will of the in the
great big tree</div>
    <div class="runofthe" id="goingto"><span id="wont">g<span
class="ow">on</span>na</span> <span id="grow"><br>gr<span
class="ow">ow</span> into</span></div>
    <a id="goingtobedinthestickysummer"
href="https://sites.pitt.edu/~dash/folktexts.html"><div
class="runofthe" id=>go<span class="ow">in</span>g</div></a>
    <div class="runofthe" id="goingtobed">Found text redacted
because it was going on for pages.</div>

    </body>
```

```
   </html>

   <!--    IMAGE SOURCES

https://www.google.com/search?q=gif+of+fish+transparent+&tbm=isch&ved
=2ahUKEwjf1OiZi8D8AhUQEFkFHTXnBtwQ2-cCegQIABAA&oq=gif+of+fish+transpa
rent+&gs_lcp=CgNpbWcQAzIGCAAQCBAeOgQIIxAnUPcLWNwlYMsnaAJwAHgAgAFIiAHO
B5IBAjE1mAEAoAEBqgELZ3dzLXdpei1pbWfAAQE&sclient=img&ei=-Pe-Y9_4GJCg5N
oPtc6b4A0&bih=738&biw=1281&hl=en#imgrc=NdicPCUD60VmoM
   https://opening.download/spring-2021.html
   TEXT SOURCE

https://en.wikipedia.org/wiki/List_of_marine_aquarium_plant_species#R
ed_macroalgae -->
```

Tutorial C: Github, Hosting, and File Organization

1. In this tutorial, we're going to learn how to make your webpages available not only in local files, but also on the WWW. To do this, we need to get your webpages **hosted** by a **server**, or a computer somewhere that serves your website up to people trying to access it online. First, choose one or more webpages you've built that you'd like to make public. (Remember that relative anonymity is the default online – so don't worry about people finding your website, unless you share your link with them!) I'll demonstrate a few different modes of creating urls; you can upload the same webpage to all of our urls, or use different webpages.
2. We're going to use Github throughout this tutorial – so, for those of you already familiar with Github, a lot of this tutorial will be redundant. First, exit this sheet and follow these instructions to upload a webpage onto the WWW using Github Pages for free. If you're making a Github account for the first time, consider that, using this mode, your username will be in your url, like so: "[your-username].github.io/".
3. You can host many webpages as extensions of the domain you just built, before Github starts to charge you. In local files and on the web, the url of any webpage is determined by the file organization. For example, if there's a directory/folder inside your "[your-username].github.io/" repository called "governor," and a webpage in that directory/folder called "big.html," the url of that webpage will be "[your-username].github.io/governor/big.html." Let's try that out. To make a directory/folder inside your repository, go to "Add file" and click "Create new file."
4. Type the name you want to give your new directory/folder (ie., what will show up in place of "governor" in the previous step). Then, type "/." Github is strange in many ways, and here's one: you need to put something in

your directory/folder when you create it. Type something random in the new blank that comes up, and then type something random into the file. Your page might look something like this:



5. Scroll down, and click “Commit new file.”
6. Now, navigate back to the main page of your repository, and you should see your new directory/folder. Click on it.
7. Once in your new directory/folder, go to “Add file,” click “Upload files,” and upload two webpages (again, these can be the same.) Make sure one is named “index.html” and the other is named whatever you want. To rename a file uploaded to Github, click on that file and then click the pencil icon to the right of “Blame.”
8. In your browser, type the url “[your-username].github.io/[your-folder-name]/” and press “Enter.” You may have to wait a few minutes or open in an incognito window – but eventually your webpage called “index.html” will show up at that url. The name “index.html” is a sort of blank or default; it’s called up by the directory it’s in.
9. Now, type the url “[your-username].github.io/[your-folder-name]/[your-second-html-page-name].html” and press “Enter.” Again, you might have to wait a few moments, but your webpage will eventually show up there! Using this organizational logic, you can easily host a whole website – we’ll talk about how to link various webpages together into a website in the next tutorial.
10. Once your page is live, it may load more slowly than it did in your local files. Various studies have been done on how long people will wait for a page to load before leaving it; you should shoot for under two seconds. For your website to load properly, your videos and images should total under 200MB (many would suggest much less, though.) Happily, there are many free online image and video compressors; just search “compress mp4,” “make gif smaller,” etc., and you’ll get tons of results. Try a few, and choose one that works best for your particular compression needs.

11. Lighter-weight pages are also best for **SEO** (**Search Engine Optimization** – or what makes your webpage show up as a Google/etc. result). We're not going to get into SEO in this tutorial, since a) people usually find web art through specific links, and b) it's a massive field of specialization, and there are always new tricks being discovered. Take a moment to google "SEO tricks" or similar, though, just to get a sense of what it entails.
12. To avoid having GitHub's stamp on your site, let's buy a domain (the cheapest are about $7/year) and use GitHub to push updates onto the site. Like your Github domain, the domain you buy can house many, many different webpages via various directories. There are many decent domain marketplaces, but I suggest Name.com, which has a user-friendly interface and lower prices than Google Domains. Before searching, look through the list of possible top-level domains — here's another good place to look. The stranger extensions won't always show up automatically – but they're sometimes the cheapest, and often look sharper. You can use Google Domains to search which domains are available (there's an option to view all extensions), and then try buying your domain from a cheaper marketplace. Search around and choose 10-15 potential domains for a work of web art you might one day make -- we'll share these in class.
13. Now, purchase one domain. It'll be a bit easier if you use Name.com, since further instructions are specific to that site – but any marketplace is fine.
14. Make a new repository in Github (from your home page, click "Repositories" and then "New"), and name it whatever you want.
15. Add a webpage into that directory (again, it's alright if it's the same webpage you added to [your-username].github.io/.) To do this, go to "Add file" and click "Upload files."
16. Click "Settings."
17. Click "Pages."
18. On dropdown menu that reads "None," select "main." Leave the next dropdown menu as-is ("/(root)"). Click "Save."
19. Under custom domain, type in your domain. Click "Save." You will get an error message, because you need to configure your domain so it's ready for Github, which we'll do next.
20. Check the box that says "Enforce HTTPS." Here's what your Github should look like (though your DNS check will likely be completed), with my example "mxeb.online":

Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the `main` branch. Learn more.

main / (root) Save

Learn how to add a Jekyll theme to your site.

Your site was last deployed to the github-pages environment by the pages build and deployment workflow.
Learn more about deploying to GitHub Pages using custom workflows

Custom domain

Custom domain

Custom domains allow you to serve your site from a domain other than `theoellinballew.github.io`. Learn more.

mxeb.online Save Remove

● DNS Check in Progress

☑ **Enforce HTTPS**

HTTPS provides a layer of encryption that prevents others from snooping on or tampering with traffic to your site.
When HTTPS is enforced, your site will only be served over HTTPS. Learn more.

21. As mentioned above, the following instructions are specific to Name.com – however, you can find similar options on different websites. When signed into Name.com, select “My domains.”
22. Click your domain name.
23. Scroll down and click “Manage DNS.”
24. You’ll need to set four A records (185.199.108.153; 185.199.109.153; 185.199.110.153; and 185.199.111.153). In all cases, leave “HOST” blank (it’ll default to your domain name) and make TTL 3600. In the end, this is what your page should look like:

TYPE	HOST	ANSWER	TTL	PRIO	ACTIONS
A	mxeb.online	185.199.108.153 GitHub Pages	3600	N/A	EDIT DELETE CREATED: 2023-02-05
A	mxeb.online	185.199.109.153 GitHub Pages	3600	N/A	EDIT DELETE CREATED: 2023-02-05
A	mxeb.online	185.199.110.153 GitHub Pages	3600	N/A	EDIT DELETE CREATED: 2023-02-05
A	mxeb.online	185.199.111.153 GitHub Pages	3600	N/A	EDIT DELETE CREATED: 2023-02-05

25. This could take up to 24 hours to register. So come back in a day and click “Check again” on Github. It’s also possible that Github will continue to show an error when your site is working – so be sure to check your url.
26. All changes in your Github files will now be reflected on your new domain. Now, using the same logic described in steps 3-9, create another url for your webpage – for example, mxeb.online/mymineralstyle/ or mxeb.online/my/mineralstyle.html
27. That’s all! Next time, we’ll turn your webpage into a connected website, and learn some Javascript tricks.

Tutorial D: File Organization and Javascript Tricks

1. Here is the promised example of what your webpage could look like at the end of this tutorial; as always, the code that makes this page is at the bottom of this tutorial. Compare if you get lost!
2. I mentioned before that a website is defined as multiple webpages that link to one another. So, first: let's turn your webpage into website by linking it to another page. We're going to use this second page to learn some very basic **Javascript**. Javascript is an extremely powerful language, used for a lot more beyond web development – here, it's especially useful in adding different kinds of interactivity. Open a new file in VSC by going to File --> New File.
3. Name your file whatever you want, using the ".html" extension – for example, "pothos.html" – and save that file in the same folder where you're keeping your index.html page.
4. Practice what you've learned already to get your new page set up. In your new file, write some bare-bones code (see step 6 on the first tutorial, and try to do it from scratch/memory).
5. Add a big chunk of found text (copy-and-pasted Wikipedia page, Q&A forum, children's story, list of data points, etc.) as a div into the body. Make this different from the text you used before!
6. Add at least two images into the body. Make these different from the images you used before!
7. Give all your elements (div and images) ids.
8. Using your ids, give all your elements some CSS – see what you can remember from last time. If you're using lots of images, of course, feel free to use classes. You can start with minimal CSS or get a little wild, making a base you're excited to animate.
9. Open your new page in your browser as you opened you index.html page in your browser: go to your new file in Finder (again – make sure it's in the same folder as your index.html), right click on it, and click "Open with [your favorite browser]."
10. Open index.html in the same way, so you have two tabs or two windows, with both pages open.
11. To link your new .html file to your index.html, you'll use **internal links**. In Tutorial B, we were working with **external links**, or links that go to pages on other websites. Internal links refer to links within one website (or that share a domain).

They are coded exactly like external links, except that you replace the url with your file name, like so:

```
<a href=[whatever-you-named-your-new-html].html>[The text or image
you want to link to your new page.]</a>
<!-- or, as an example: -->
<a href=pothos.html>Figures long not to</a>
```

In my example, the words “Figures long not to” will link to my second page.

12. Make your index.html webpage link back to your new webpage, so that each links to the other via at least one element (remember, links can be texts or images.)
13. In the last tutorial, we indicated links by making elements change when you hover over them, via :hover pseudo-class selectors. Also useful for indicating a link is the "cursor" CSS property, which changes the image of your cursor based on the element you're hovering over. Like the :hover pseudo-class selector, the cursor property can be defined for any element – not only links. However, since these two modes (a change in cursor and a change in the element being hovered over) are the most standard modes of defining links, I recommend being careful about how you deploy them for non-links. Even in web art, a user doesn’t feel great if it appears that they should click, but nothing happens when they do! Let’s first set a standard cursor for your page by defining the cursor property in the CSS for your body, just as we defined a background color for your index.html page:

```
body {
      background-color: yellow;
      cursor: vertical-text;
      }
```

14. This page gives a full list of possible cursor values. Replace “vertical-text” with some of those values, and choose the one you want for your body default.
15. You can probably guess how to change the cursor for a given element: you add the cursor declaration to the CSS of that element. For example, if you want the user's mouse to show up as a question mark when they're hovering over an element with id=arewethereyet, you would add the following CSS:

```
#arewethereyet {
  cursor:help;
}
```

Of course, you'll also have other CSS declarations for #arewethereyet. Give a cursor value to at least two of the elements on your page, links or otherwise.

16. Before we get started on Javascript, a note on organization: as your website gets larger and more complex, you may want to put all your images into a folder within the folder where you've been keeping your code. If you do, the **file paths** (or that which goes after "src=") will just be a little more complicated:

```
<img
src="[the-name-of-the-folder-where-you-put-your-image]/[your-image-name.extension]">
```

17. Now, you can close your index.html, and focus on your new webpage.
18. Let's start by writing some Javascript directly into your HTML. Edit the opening tag for your div so that it looks like this:

```
<div onclick="innerHTML = Date()" id="[your-div-id]">
```

19. After saving and refreshing, try clicking your div – when you do, it should be replaced by a very detailed version of the date and time. Congrats - you've written and executed your first Javascript!
20. For the rest of this tutorial, though, we'll keep Javascript separate from HTML. Javascript code – or **scripts** – should be written at the bottom the body of your code (right before </body>). To start, write opening and closing tags like so:

```
<script></script>
```

21. Everything between those two tags will be in Javascript. VSC detects that you're writing in Javascript here. Write something in between these tags, highlight it, and comment it out, using "command" or "control" + "/". Notice that VSC automatically adds different punctuation for commenting out here than in HTML or CSS.
22. Now, add the following in between <script> and <script>:

```
            window.onload = function() {
                document.getElementById("[your-div-id]").innerHTML = Date();
            }
```

23. When you save and refresh, the text of your text div should be replaced by a very detailed version of the date and time. Let's break down what just happened.
    - The first line above tells the browser that everything between the "{" and "}" should happen (**function**) as soon as the webpage (**window)** loads (**onload**).
    - The second line tells the browser to call up an element on this page by its id (**document.getElementById)**. It then says to change the text of that element (**innerHTML**) to the date – a **built-in object**.
24. "Date()" can also be replaced by any text you wish, if you put that text in quotes. Try that.
25. Let's do something a bit more exciting. Replace "Date()" with the following:

```
wINdo[Math.floor(Math.random() * wINdo.length)
```

26. Then, add the following code inside your script (between <script> and </script>) – before or after all the code you've already written:

```
            let wINdo = [ "vomiting leaf", "wandering trill" , "good
dull filler", "really thick curve", "winnowy gullet", "random
hearth", "full belly", "orange", "loud keyboard", "calculating turn",
"burgeoning chest", "zinger"]
```

27. To review, your whole script should look like this:

```
<script>
            window.onload = function () {
            document.getElementById("[your-div-id]").innerHTML =
wINdo[Math.floor(Math.random() * wINdo.length)];
            }

            let wINdo = [ "vomiting leaf", "wandering trill" , "good
dull filler", "really thick curve", "winnowy gullet", "random
hearth", "full belly", "orange", "loud keyboard", "calculating turn",
"burgeoning suitcase", "zinger"]

</script>
```

28. Save and refresh – and then refresh again, and again. You'll see that your div is being replaced by texts from the list defined as "wINdo" chosen at random. Now, try moving that list (everything from "let" to " "zinger"]") to the very top of your script – or right after <script>. When you refresh the page, you'll see that nothing

has changed; the order of these components doesn't matter. You could even have these in separate scripts, and it wouldn't make a difference.

29. I won't get into the details of what's happening with Math here, but this is the code you can use to randomize something (almost anything). Now, throughout your code, change "wINdo," which is just your private name for your list or **variable**, to something else of your choosing. Like other names, have fun with it – but with Javascript there are different naming rules:
    - no hyphens (throughout Javascript, terms that use hyphens in HTML are often replaced by **camel case** – for example, fontSize replaces font-size)
    - underscore, dollar sign, digits, letters are all allowed
    - must begin with a letter
    - no reserved words
30. Now, rather than replace your entire div, let's replace a small section of your div (one word, phrase, or sentence), using a span – and make that happen when you click on the larger div. First, make one section of your div into a span with an id – if you forget how to do this, look back to Tutorial B.
31. In your script, replace the id of your div with the id of your span.
32. Now, replace "window.onload" with "document.getElementById("[your-div-id]").onclick". Both "onload" and "onclick" are **event handlers** and help Javascript execute some action when the user does something. Now, your span will randomize when you click on your div, rather than when the page loads. Here's that code:

```
document.getElementById("[your-div-id]").onclick = function () {
            document.getElementById("[your-span-id]").innerHTML =
wINdo[Math.floor(Math.random() * wINdo.length)];
        };
```

33. Feel free to take a moment to play; already there are many things you can do by rewriting the code I've already given you. You can change all the items of my example list, so that every option makes some kind of meaningful sense (or strange nonsense) in the context of your larger text. You can have any number of items in your list.
34. Previously, we learned how to change the CSS of an element when you hover on it, using pseudo-class selectors. Now, let's use Javascript to change the CSS of other elements when you hover over an element. Add the following into your script:

```
document.getElementById("[your-div-id]").onmouseover = function () {
            document.getElementById("[your-span-id]").style.color =
```

```
"blue";

document.getElementById("[your-span-id]").style.backgroundImage =
"conic-gradient(green, red, yellow, red, yellow, green)";
            document.getElementById("[your-img-id]").style.opacity =
".5";

document.getElementById("[your-second-img-id]").style.display =
"none";
        };

document.getElementById("[your-div-id]").onmouseout = function () {
            document.getElementById("[your-span-id]").style.color =
"red";

document.getElementById("[your-span-id]").style.backgroundImage =
"none";
            document.getElementById("[your-img-id]").style.opacity =
"1";

document.getElementById("[your-second-img-id]").style.display =
"block";
        };
```

Above, I've asked the browser to complete three functions when the user hovers over the div; you can call as many functions in this way as you want. You can do this with just about any CSS – replacing "color" in the first line with "backgroundColor," "borderRadius," or almost any other property we've learned – including the value in quotes, as with "blue". In most cases, a hyphen in a property will be replaced by a capitalization; ie., "border-radius" in CSS becomes "borderRadius" in Javascript; you do not have to do anything different for the value, though. (For more info on CSS possibilities, use the dropdown menu here.)

35. Play! You now know how to make any element(s) appear/disappear and/or change in any way when you hover or click on any other element(s). Note that you can randomize not only words, but also property values for CSS – as I did in the example (see code below). You'll also see below how you can use "this" to indicate that a function is meant to happen to the same element that your mouse is hovering over:

```
let allcolors = ["blue", "green", "yellow", "red", "black", "white",
"pink", "purple", "navy", "orange", "turquoise"]

document.getElementById("full").onmouseenter = function() {
               this.style.backgroundColor =
allcolors[Math.floor(Math.random() * wINdo.length)];
         };
```

36. As you add more and more elements and functions, you may want to move your Javascript to another file. To do so, make another new file in VSC named whatever you want – but this time with the ".js" extension – for example, "philodendron.js".
37. Save your new file in the same folder as your index.html webpage and your new webpage. To review, here's what your website folder should currently contain:
    - index.html
    - [whatever-you-named-your-new-html].html
    - [whatever-you-named-your-js].js
    - any images or videos you've included in either of your pages
38. Now, cut everything from in between your script tags and paste it into your new .js file. Your opening and closing <script> tags should remain in the HTML file – but you should add "src = "[the-name-of-your-js-file].js" ", like so:

```
 <script src="C.js"> </script>
```

39. Now, go wild. Experiment with anything and everything, and, as always, push the code. Make more pages if you need/want to, and more internal links linking them together.

Here's the HTML code [again, with the long div clipped]:

```
<!DOCTYPE html>
<html>
<head>
   <style>
       body {
            background-color: azure;
            cursor: vertical-text;
            }

       .fullones {
```

```
            position: fixed;
            z-index:10;
        }
        #full {
            bottom:30vh;
            border-radius: 50%;
            width: 30vw;
            left: 35vw;
            z-index: 10;
         }


         #fuller {
            width: 12vw;
            right: 0vw;
            bottom: 30vh;
            display: none;
         }

        #goingtobed{
            position: relative;

        }
        .flippers {
            color: red;
        }
        </style>
</head>


<body>
<img class="fullones" id="full" src="tutDimg/earth1.png">
<img class="fullones" id="fuller" src="tutDimg/earth2.png">
<div id="goingtobed">The algae, <a href="B.html"><span
class="flippers" id="agar">Agardhiella</span></a>, has similar
requirements to Gracilaria and Ulva and features translucent,
gelatinous fronds. In captivity, it can tolerate, but must first be
slowly introduced to bright lighting and can be fed to fish. It is
```

```
also a source of the substance, agar.[133][134]
   This reddish purple, gelatinous algae is characterized by its
numerous branchlets. It is edible to humans. Once known as
Agardhiella tenera.[13][136]
 This is <br><br><br>

   a hardy species suitable for marine aquaria. It develops darker
<span class="flippers" id="color">coloration</span> under lower
intensity light and is best attached to rockwork with superglue.

<script src ="D.js"></script>
</body>
</html>
```

…. And here's the Javascript code:

```
let wINdo = [ "vomiting leaf", "wandering trill" , "good dull
filler", "really thick curve", "winnowy gullet", "random hearth",
"full belly", "orange", "loud keyboard", "calculating turn",
"burgeoning chest", "zinger"]

let allcolors = ["blue", "green", "yellow", "red", "black", "white",
"pink", "purple", "navy", "orange", "turquoise"]

window.onscroll = function () {
           document.getElementById("agar").innerHTML =
wINdo[Math.floor(Math.random() * wINdo.length)];
           document.getElementById("color").innerHTML =
wINdo[Math.floor(Math.random() * wINdo.length)];
       };

document.getElementById("goingtobed").onclick = function() {
   document.getElementById("agar").innerHTML =
wINdo[Math.floor(Math.random() * wINdo.length)];
   document.getElementById("color").innerHTML =
wINdo[Math.floor(Math.random() * wINdo.length)];
};
```

```
document.getElementById("full").onclick = function () {
   document.getElementById("fuller").style.display = "block";
}


document.getElementById("full").onmouseenter = function() {
           this.style.backgroundImage = "url('tutDimg/earth3.png')";
           this.style.opacity = ".6";
           document.getElementById("goingtobed").style.color = "red";
           this.style.backgroundColor =
allcolors[Math.floor(Math.random() * wINdo.length)];
           document.getElementById("goingtobed").style.border = "red
dotted 60px";

document.getElementById("goingtobed").style.backgroundImage =
"url('tutDimg/earth3.png'), url('tutDimg/earth2.png')";
         };

document.getElementById("full").onmouseleave = function() {
           this.style.backgroundImage = "none";
           this.style.opacity = "1";
           document.getElementById("goingtobed").style.color =
"black";
           this.style.backgroundColor =
allcolors[Math.floor(Math.random() * wINdo.length)];
           document.getElementById("goingtobed").style.border =
"purple dashed 60px";

document.getElementById("goingtobed").style.backgroundImage =
"conic-gradient(green, red, yellow, red, yellow, green)";
         };
```

Tutorial E: Responsive Web Design, Accessibility, and Other Resources

1. This tutorial (the final one!) will focus on a wide range of topics. We'll start by learning to build **responsive websites** – or websites that function well on various window sizes and devices. Then, we'll scratch the surface of **accessible web design** – or optimizing sites for those who are differently-abled. I'll then give a few suggestions for how to continue to learn to code on your own. Here's your example of a responsive and somewhat accessible site; the code is below. Before we get started, choose a site you've been working on that you feel is near-complete or complete. This may be your midterm, or an early draft of your final.

2. First, go to some big tech sites or major online news outlets, sites where you know tons of $$$ has been poured into development – and start adjusting the size of your browser window. Notice how the elements shift as your browser window gets bigger or smaller – and also that, often, when the screen is a certain size, something about the page shifts drastically. You will probably also notice that, even for these big money websites, there are some screen sizes that just look bad, or even have some legibility issues. Below, we'll talk about how to effect both these incremental shifts and the more drastic ones (which are often used for optimizing for various browser sizes and devices).

3. Let's start by making a site function perfectly on desktop for reasonable window sizes. The first trick – which you may have figured out already – is to position each element according to the corner it is nearest. For example, if the link to your next page is in the bottom-right-hand corner, you want to position it using "bottom" and "right" – rather than "top" and "left." If you position it using "top" and "left," it may not stay in the bottom-right-hand corner for different window sizes. In the sample code, I did this with the screenshot buttons (#ss). Before we get started, make an empty div with id="bottomright".

```
<div id="bottomright"></div>
```

4. Now, add the following into your CSS:

```
#bottomright {
position: fixed;
background-color: blue;
width: 30px;
height: 30px;
bottom: 30px;
right: 30px;
z-index: 10000;
}
```

5. Make sure you have a little blue square in the bottom-right-hand corner of your window, and adjust the window to make sure it stays there at all times. Note: if you have other elements with a higher z-index than 10000, you may need to give this div a higher z-index to make sure it shows up.

6. Another good rule of thumb is to use measurements that relate to the window size. We discussed this in Tutorial A but some measurements are fixed (like pixels), whereas others are relative to the window size. We learned about vw, which corresponds to the percentage of the viewport width, and vh, which corresponds to the percentage of the viewport height. Try replacing the code above with the following, and see what happens when you adjust your window size:

```
#bottomright {
position: fixed;
background-color: blue;
width: 9vw;
height: 9vh;
bottom: 3vh;
right: 3vw;
z-index: 10000;
}
```

7. Neither of the above options is necessarily better or worse; depending on the look you're trying to achieve, one or the other might be better. Vh/vw measurements can be used for most properties in CSS. Using vh/vw can also allow you to side-step step three – which can, in some cases, be easier or more logical. Try changing the code to the following, and see what happens when you adjust the window:

```
#bottomright {
position: fixed;
background-color: blue;
width: 9w;
height: 9vh;
top: 97vh;
left: 97vw;
z-index: 10000;
}
```

8. Play around with adding vh/vw measurements throughout your code. There's no right way to do this – maybe you always want a certain image to be exactly the same size or exactly the same distance of a viewport side no matter what size the viewport is. That's fine; use pixels! For legibility reasons, people often use a fixed measurements like pixels for font-size.

9. Sticking with the example above, let's say that you want your blue rectangle to have a width of 9vw, but you never want it to be smaller than 90px or larger than 300px. This is actually really easy; we'll just use "min-width" and "max-width":

```
#bottomright {
position: fixed;
background-color: blue;
width: 9vw;
min-width: 90px;
max-width: 300px;
height: 3vh;
top: 97vh;
left: 97vw;
```

```
z-index: 10000;
}
```

10. There's also "min-height", and "max-height". I often determine heights and widths with vw/vh – and add minimums and/or maximums for windows of more extreme sizes. Play around with this! With these tools alone and some creativity, you should be able to make a website that looks good with all reasonable window sizes on desktop (don't worry about extra small or extra larges sizes just yet.) Remember that, when this project is out in the world, you will not be able to stand beside your reader/viewer/user and tell them how big their window should be.

11. Now, let's learn how to give specific CSS to a mobile site. First, you'll want to make sure your website is registering the width of the specific device you're using. Add the following into the head of your code (but NOT in the CSS):

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

12. I mentioned that, when adjusting the windows of existing sites, a lot may change very suddenly. These sites are probably using **media queries**. Media queries tell the browser to use different code for different screen widths and many coders use them to optimize their sites for mobile devices. This means that a person on their laptop can see something very similar to what a person sees on their phone, if they adjust their window to be the size of a phone. Add the following into your CSS:

```
@media (max-width: 600px) {
      body {
      background-color: red;
      }
}
```

13. Now, try adjusting your window size till it's about the size of your phone. Once the width dips below 600px, the background of your window should turn red.

14. Any CSS between the brackets above will only run when the width of the screen is below 600px. It will also override anything in the rest of your CSS – so don't

worry about contradicting yourself. 600px is a standard measurement for designating cellphones, but you can choose any amount. Now, try making your blue rectangle be on the bottom-right-hand corner on desktop, but jump to the top-right-hand corner for mobile.

15. Now, say you need a whole new element on your mobile site that doesn't exist on desktop. This is often the case – say you want a different version of a logo/image for desktop and mobile, or a graphic that toggles a menu on and off. Let's stick with our blue rectangle, here. In the non-media-query CSS, add the following declaration:

```
display: none;
```

16. Then, add the following into your media query:

```
@media (max-width: 600px) {
      #bottomright {
      display: inline;
      }

      body {
      background-color: red;
      }
}
```

17. When you adjust your window, your blue rectangle should show up only when the width is below 600px. You can also do display:block – while "inline" will make your element act as a would, "block" will make your element act as a would. In this case, there shouldn't be a difference.

18. Before you play more! Note that adjusting window size might not give an accurate sense of what your mobile version will look like. To get another view (on a Mac), go to View → Developer Tools → Inspect Elements.

19. Click on the little icon in the top let of the developer tab that looks like this:

20. There's a dropdown menu that lists various devices. When you select a certain device, your browser will try its best to generate an example of what your site would look like on that device. A word of caution: even this method is sometimes faulty – and not always more accurate than adjusting your window. So don't get too excited until you push your site onto Github and check on your mobile device.

21. Now, try doing a lot more in the media query space. There will often be a lot of code in this space. Don't forget about the tools already mentioned in this tutorial, especially min/max-height/width!

22. You can make multiple media queries. Some people use media queries when making their desktop version responsive. A standard media query for a tablet is:

```
@media only screen and (min-width: 768px) and (max-width: 1024px){

}
```

23. Feel free to do play with adding specific CSS for a tablet version!

24. And yes, thank goodness: you can also specify Javascript for different viewports. Add the following script inside your body, at the bottom:

```
<script>
const isComp = window.matchMedia('(min-width: 600px)')

if (isComp.matches) {
alert("you're probably on your desktop!");
}

</script>
```

25. Now, if you load your website on desktop, assuming your width is larger than 600px, you should get an alert that says “you're probably on your desktop!” The term “isComp”, here, is just what I chose to name the constant – you can replace it with anything you want, as long as you replace it in both places. Any Javascript between the two brackets {} above will only execute when the width of your screen is at least 600px. Probably some of the Javascript you’ve already written (including anything that responds to mouse position or hover – things that don’t exactly exist on the phone) should only execute for desktop. Make sure that code lives in between the two brackets in the code above.

26. Conversely, maybe there’s some Javascript you only want to execute for the mobile version of your site. You can probably figure out how to write that code, but here it is just in case:

```
<script>
const isPhone = window.matchMedia('(max-width: 600px)')

if (isPhone.matches) {
alert("you're probably on your phone!");
}

</script>
```

27. Now, let’s talk about **accessibility** – or making your work available to those who are differently-abled. This is a huge, ever-expanding, exciting field. There are so many ways of being differently-abled, and so many ways to creatively address those differences. Here, we’ll just cover some common accessibility tools. I won’t always go into detail about these processes, some of which are quite involved. However, this should at least give you a sense of the work that can and should, ultimately, be done in this realm. Of course, some of this will not apply to your specific web art projects, and that’s okay – it’s good to know for the future, and also (hopefully) can inspire you to start thinking creatively about other ways to make your works of web art accessible.

28. Let’s cover accessibility for those with vision impairments first – since the web is a primarily visual medium. Most of this work involves learning how to design for

s**creen readers,** which convert text on a website into audio. Screen readers rely heavily on an element we have not discussed: **headers**. Headers are textual elements with preset CSS – so they are often more annoying than useful when making web art. If your text is informational, though, you should use headers to outline your information clearly, so those using screen readers can better parse it. Here's an example of how headers appear. You can and should give specific CSS to your headers, overriding their default CSS. You should not skip levels (ie, jump from <h1> to <h3>), which may cause confusion.

29. Also, all images should have **alt text**, or text that describes the image. Alt texts will also be read out loud by screen readers. There's a really wide range of alt texts out there. Many people suggest keeping the alt texts short and avoiding "image of," which can get repetitive – knowing that all this text will be read out loud. For a work of web art, though, I encourage you to get creative with these – this is your chance to describe exactly how your reader should be experiencing the images you've included. Below, I've included long descriptions because I think the visual aspect of the site is very important:

```
        <img  alt="One of 5 gifs of frames that act as borders for the webpage and layer on top
of each other, shuffling when you scroll. This one is rainbow-colored and recalls the 70s and
natural gooey movement. It also has a solid and straight non-animated blue-grey line around
its edge." id="rainbow"  class="borders" src="frames/rainbow.gif">


        <img  alt="One of 5 gifs of frames that act as borders for the webpage and layer on top
of each other, shuffling when you scroll. This one has as its background a black curved thick
line, something like what a child would draw. On top of that are about twenty images
iterations of one flower, moving in various sizes and positions. The flower has 5 petals,
darker pink on the edges, and a yellow center with little stems coming out of it. The petals
are far apart from one another." id="pink" class="borders" src="frames/pinkflowers.gif">


        <img  alt="One of 5 gifs of frames that act as borders for the webpage and layer on top
of each other, shuffling when you scroll. This one also has a solid and straight non-animated
blue-grey line around its edge. Here, the animated part consists of bright pink and dark
magenta stripes arranged as though they are coming out from the center as rays from a sun.
They are speckled with white sparkles of various sizes. This is one of two gifs that are not
rectangular; the flowers stick out beyond the black border, and their edges are visible no
matter how the gifs are layered." id="borderp" class="borders" src="frames/borderpink.gif">
```

```
        <img  alt="One of 5 gifs of frames that act as borders for the webpage and layer on top
of each other, shuffling when you scroll. This one is all bright computational green and
black. It resembles the rainbow one in its movement: there's something liquid about it, like
oil in water -- where the borders between substances are curved and also at times blur into
one another. But it's much slower than the rainbow one, and the chunks of green and black are
wider." id="slowg" class="borders" src="frames/slowg.gif">


        <img  alt="One of 5 gifs of frames that act as borders for the webpage and layer on top
of each other, shuffling when you scroll. This is one of two gifs that are not rectangular;
while the outer edge is rectangular and flush with the edges of the window, the inner edge is
very complex. It's the most complex of all the gifs. On the outer side it has a light green
and black plaid pattern. On top of that and toward the center is a shimmering green and black
design that recalls both ornate antique frames and video game depictions of turbulent seas.
Elements of this are visible in the corners no matter how the gifs are shuffled." id="green"
class="borders" src="frames/green.gif">
```

30. Alt texts will also display if the link to your image is broken for whatever reason. To see this in action, try changing the file name of your image, without changing the src in your code – when you reload, you'll see that that image (the link to which is now broken) is gone and the alt text is in its place.

31. For those with hearing differences, you'll want to make sure any important audio text is transcribed and available somewhere on the website. There are many different free online tools that will draft a transcription of an audio for you – but you should definitely plan to edit that text yourself. Depending on the length of your audio, you may choose to summarize it in the transcription. Videos are more complex, as you will ideally include subtitles.

32. Many people with mobility differences operate websites via only the keyboard (tab and enter keys) or other devices that correlate to those keys. You need to make sure this is possible! A good resource for learning how do this is here.

33. A lot can be done with the visuals alone. First, any informational text should be "legible" in a canonical way (NOT how we discussed in class): the text should be big, in a legible font, and with a large contrast with the background. Also, any site that includes flashing lights or intense visual content should have a warning.

34. To learn more about accessibility measures, take a look at this authorized Web Accessibility Initiative summary.

35. Finally, let's talk about how to continue on your code-learning journey. This should go without saying, but googling a question will often turn up good results. Examples often help me most, and Codepen is especially good, since you can alter the sample code in the browser.

36. For a less problem-oriented approach, I recommend W3 Schools and MDN Web Docs, both of which clearly outline a lot of different capabilities of the code and can take you step by step.

37. If you are trying to turn coding into a career, your next step should probably be learning React, which is currently the most prevalent of the **Javascript libraries and frameworks** (both of which add functionality and ease when coding with Javascript). React was developed by Meta, which is unfortunate, but it will make you more eligible for jobs than any other library or framework.

38. Thanks for running through this tutorial! Always feel free to reach out if you have any questions.

Here's all the code for the responsive and somewhat accessible example site:

```
<!DOCTYPE html>
<htmL>
   <head>
       <meta name="viewport" content="width=device-width, initial-scale=1">
       <link rel="preconnect" href="https://fonts.googleapis.com">
       <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
       <link
href="https://fonts.googleapis.com/css2?family=Cinzel&family=Cormorant&family=Cormoran
t+Garamond&family=Labrada:wght@100;200&family=Noticia+Text:ital@0;1&family=Playfair+Di
splay&family=Poltawski+Nowy&family=Space+Grotesk:wght@300&family=Spectral:wght@200&dis
play=swap" rel="stylesheet">
       <style>
           body {
               font-family: 'Cormorant Garamond';
               line-height:18px;
               font-size: 15px;
```

```
}

::-webkit-scrollbar {
     display: none;
}

.wholes {
    width: 30vw;
    top: 0vh;
    left: 0;
    position: fixed;
    text-align: justify;
}

#ss {
    position: fixed;
    bottom: .6vw;
    right: 3.3vw;
    z-index: 200;
    white-space: pre;
}

.sss:hover {
  text-decoration:underline;
}

.sss {
    background-color: white;
}

#pink {
    position: fixed;
    top: -12vh;
    left: -12vw;
    width: 124vw;
    height: 124vh;
}

#rainbow, #creepg, #sixth {
    position: fixed;
    top: -3vh;
    left: -3vw;
```

```
            width: 105vw;
            height: 105vh;
        }

        .borders {
            position: fixed;
            top: 0;
            left: 0;
            width: 100vw;
            height: 100vh;
        }

        .back {
            position: relative;
            height: 2400vh;
            width: 100vw;
            opacity:.0001;
            z-index: 100;
        }


    @media (max-width: 600px) {
        .borders {
            display: inline;
        }

        #ss {
            position: fixed;
            bottom: 1.5vh;
            right:0;
            z-index: 10;
            white-space: normal;
            width: 100vw;
            text-align: center;
        }
        .wholes {
            position: relative;
            width: 60vw;
            left: 18.5vw;
            top: 45vh;
            text-align: justify;
        }
```

```
            #wholec {
                padding-bottom: 45vh;
            }

            .back {
                display:none;
            }
        }

        </style>
    </head>

    <body  onload=flip() onscroll="flip(); gifs()">
        <div class="wholes" id="wholea">(A) how we treated my <span id=nouna>vomiting
leaf</span> till it was a<span id=nounb>sycophant</span>: we <span id=verba>tossed
it</span>, <span id=verbb>tickled it</span>, set it <span id=adja>free</span>; till it
<span id=verbc>rocked</span> our whole world <span id=adjb>flat</span>; till it <span
id=verbd>knocked open</span> our <span id=nounc>done topped earth</span> and... we
<span id=verbe>ripped</span> all our <span id=nound>petals in shreds</span>
</div><br><br>
        <div class="wholes" id="wholeb">(B) where did our <span id=grand>grand
stuck</span> <span id="bird">holy bird</span> <span id="go">go</span>&mdash;after we
<span id=riled>riled up</span> its <span id="mutters">mutters</span> and stacked its
<span id="goats">turning goats</span>; did it <span id="rush">rush through its
furs</span> and leave for <span id="good">good</span>? or did it <span id="rip">rip
up</span> our <span id="hearts">soft and weak hearts</span>, meeting <span
id="horizon">a rippled horizon</span></div><br><br>
        <div class="wholes" id="wholec">(C) inside our <span id='train'>whole big
train</span> every <span id='shorts'>short skirt</span> <span id='filed'>tucked
under</span>, and then found our <span id='past'>flowing past</span>; we didn't <span
id ='feel'>feel</span> the <span id ='trees'>trees</span>; we watched <span
id=rain>the rain</span> <span id=flow>flow</span> our bellies and helped the <span
id=swans>swans</span> as they <span id=rocked>rocked side to side</span> and our
thoughts didn't turn out the way we'd hoped but they <span id=were>were</span> enough
to <span id=hold>hold</span> us</div>
        <div class="back"></div>

        <img  alt="One of 5 gifs of frames that act as borders for the webpage and
layer on top of each other, shuffling when you scroll. This one is rainbow-colored and
recalls the 70s and natural gooey movement. It also has a solid and straight
```

```
non-animated blue-grey line around its edge." id="rainbow"  class="borders"
src="frames/rainbow.gif">
       <img  alt="One of 5 gifs of frames that act as borders for the webpage and
layer on top of each other, shuffling when you scroll. This one has as its background
a black curved thick line, something like what a child would draw. On top of that are
about twenty images iterations of one flower, moving in various sizes and positions.
The flower has 5 petals, darker pink on the edges, and a yellow center with little
stems coming out of it. The petals are far apart from one another." id="pink"
class="borders" src="frames/pinkflowers.gif">
       <img  alt="One of 5 gifs of frames that act as borders for the webpage and
layer on top of each other, shuffling when you scroll. This one also has a solid and
straight non-animated blue-grey line around its edge. Here, the animated part consists
of bright pink and dark magenta stripes arranged as though they are coming out from
the center as rays from a sun. They are speckled with white sparkles of various sizes.
This is one of two gifs that are not rectangular; the flowers stick out beyond the
black border, and their edges are visible no matter how the gifs are layered."
id="borderp" class="borders" src="frames/borderpink.gif">
       <img  alt="One of 5 gifs of frames that act as borders for the webpage and
layer on top of each other, shuffling when you scroll. This one is all bright
computational green and black. It resembles the rainbow one in its movement: there's
something liquid about it, like oil in water -- where the borders between substances
are curved and also at times blur into one another. But it's much slower than the
rainbow one, and the chunks of green and black are wider." id="slowg" class="borders"
src="frames/slowg.gif">
       <img  alt="One of 5 gifs of frames that act as borders for the webpage and
layer on top of each other, shuffling when you scroll. This is one of two gifs that
are not rectangular; while the outer edge is rectangular and flush with the edges of
the window, the inner edge is very complex. It's the most complex of all the gifs. On
the outer side it has a light green and black plaid pattern. On top of that and toward
the center is a shimmering green and black design that recalls both ornate antique
frames and video game depictions of turbulent seas. Elements of this are visible in
the corners no matter how the gifs are shuffled." id="green" class="borders"
src="frames/green.gif">
       <div id="ss">
                   <span class="sss" onclick = ssa()>screenshot (A)</span>
             <span class="sss" onclick = ssb()>screenshot (B)</span>
       <span class="sss" onclick = ssc()>screenshot (C)</span></div>

       <script>
           let allz = ["3", "4", "5", "6"]

           function gifs() {
```

```
                document.getElementById("rainbow").style.zIndex =
allz[Math.floor(Math.random() * allz.length * 6)];
                document.getElementById("pink").style.zIndex =
allz[Math.floor(Math.random() * allz.length * 6)];
                document.getElementById("borderp").style.zIndex =
allz[Math.floor(Math.random() * allz.length * 6)];
                document.getElementById("green").style.zIndex =
allz[Math.floor(Math.random() * allz.length * 6)];
                document.getElementById("slowg").style.zIndex =
allz[Math.floor(Math.random() * allz.length * 6)];
            };

            const varnouna = [ "vomiting leaf", "wandering trill" , "good dull filler",
"really thick curve", "winnowy gullet", "random hearth", "full belly", "California
orange", "loud keyboard", "calculating turn", "burgeoning tit", "zinger head"]
            const varnounb =[" sycophant", " cinder block", " thick cough", "n
uncovered wing", " windy fly", " bird in sickness", " ridged witness", " triumph", "
rotten plow", " cheated glimmer", " grandiose and violent text", "n email sent from
the ground", "n indeterminate stat"]
            const varverba = ["tossed it","rigged it to a school chair", "riled it up",
"filled it with cream", "drew closed its shutters", "opened its mouth", "filled its
gut", "rounded its corners", "called its mother", "texted its brother", "filled it
in"]
            const varverbb = ["wrapped its wound", "wrapped its candies", "glittered
its cheeks", "filed away its energies", "scraped its hard spots", "grew its tongues",
"wronged its ears", "gilded its poles", "compiled its stats", "gave it a tip", "looked
it up"]
            const varadja = ["free", "swinging", "tilted", "pretty", "true",
"perched","still", "rocking", "rolling", "spinning", "in gold", "on a table", "into a
hole", "onto the wind", "up", "in stone", "into breath", "down"]
            const varverbc =["rocked and rolled", "ground down", "minced", "ran",
"coughed up", "curled", "shook", "plunged", "time-traveled"]
            const varadjb = ["into the sea", "so it became purple", "into my heart",
'onto astroturf', 'into my little-girl canned soup', "toward America", "on " +
Date(),"into the foliage", "across my hip", "toward the sleepless morning hours",
"through heartbreak", "past sleeplessness", "past heartbreak", "through illness,
health", "past tragedy"]
            const varverbd = ["knocked open", "locked shut", 'stuffed full', "rippled",
'dragged to death', 'hotglued together', 'had sold off every speck of','ad libbed',
'had talked through', 'had devoured', 'flung out', 'had filed down', 'crowned', 'broke
down', 'garbled', 'had wrung out']
```

```
            const varnounc = ['topped-off soil','red robin', 'many red belts', 'two
great guests', 'itsy week', 'deflating breast', 'long wait', 'interim boss',
'unacceptable wages', 'broken broken thing', 'empty morning hours without the one we
love', 'realism', 'wretched moth-eaten toys', 'morning glory']
            const varverbe = ['remembered', "couldn't weather", 'fell into', 'called
up', 'gave a break to', 'got lost in', 'thought back to', 'reflected back', 'sunk
into', 'wrapped arms around', 'gave up on' ]
            const varnound = ['ferns in shreds', 'faces in dirt', 'little rains',
'thick inner sides', 'tired propagations', "car parts (torn off our beloved car)",
'truncated mournings', 'refusals of sadness', 'last resources for sadness', 'grand
imaginings', 'tired tree-lined honors', 'shoulder-length thoughts']

            const c_grand = ["grand stuck", "thick", "rounded and hurt", "tired",
"grown-up", "temporary", "love-bombing", "interested", "unnamed and wretched", "sad
and sunken", "soft and salted", "midnight"]
            const c_bird = ["holy bird", "(foaming) drink", "(holy) milk", "(preaching)
gut", "credit line", "(fuller) gut", "blue-light-resistant glass pane", "headphone",
"(fucked) brain", "gluten-free heart"]
            const c_go = ["go", "slip off to", "fill itself", "reach its inner light",
"get so uptight", "suck its ginger", "dry out its skin", "inspire masses", "fold its
hands", "run through the grasses", "read into our hearts"]
            const c_riled = ["riled up", "sold", "sent out", "tucked away", "skimmed",
"graphed", "overturned", "translated", "inserted ourselves into", "crowned", "looked
into the heart of"]
            const c_mutters = ["muttered words", "silent goals", "hair nets", "knitted
loves", "hot hot drinks", "inner fuzz", "water-filled glove", "flipped horizon",
"eyes", "hairline"]
            const c_goats = ["spoiled gold", "fingers", "soda cans", "plastic blue
cups", "borrowed broken souls", "memories", "dreams", "sand papers", "bedsheets", "old
solid flasks", "buildings", "eye masks", "computers"]
            const c_rush = ["rush through its furs", "garble roads", "cut its chin",
"wrap up its books", "flip through our photos", "load up its life", "upload some
tracks", "initiate a talk", "board a flight", "blow into a tube", "divide its own
private number"]
            const c_good = ["another land", "dirty love", "better or for worse",
"good", "once", "the greater good", "us", "a better world", "what's greener", "the
mending of its inner child"]
            const c_rip = ["rip up", "bring into itself", "guzzle down", "regurgitate",
"breathe life into", "re-inflate", "muscle through", "lift up", "peer into", "trip
up", "enter into", 'steal']
```

```
            const c_hearts = ['soft and weak hearts', '"bubbly" hearts', 'immaculate
code', 'very own thick clouds', 'dearest pillow', 'very still waters', 'earliest
thoughts', "gravest tongues", 'single language']
            const c_horizon = ['the sea', 'a god', 'breath', 'its own rippled line',
'ruby red', 'its own apostrophe', 'dissolve', 'the outer great great past', 'a
once-gone feeling', 'a less obvious land', 'ground']

            const c_train = ['truest trailer', 'massive purse', "long train", 'hundreth
house', 'private war', 'insect cloud', 'scavenged mirror', 'forest', 'ugly printout',
'biggest city']
            const c_shorts = ['skirt scrap', 'great hope', 'piece of flower', 'lost
piece of punctuation', 'byte', 'damp laugh', 'sexy lump', 'saltine-speckled wrist',
'dry throat']
            const c_filed = ['was tucked away', 'had a blast', 'got wasted', 'went
unheard', 'tried to speak', 'got trampled', 'swallowed', 'played and won a game',
'filled a cup', 'wandered']
            const c_past = ['flowing past', 'private mint toothpaste', 'innermost
froth', 'farthest streetlight', 'event horizon', 'treasure chest', 'secret
transpiler']
            const c_feel = ['feel', 'hold onto', 'read into', 'get into', 'walk past',
'go through', 'get a grasp of', 'eat', 'hear', 'get through to']
            const c_trees = ['trees', 'different genres of gravel', 'harder walls',
'hollow walls', 'piles of brick', 'piles of tiles', 'categorized jewelry', 'marble
counter samples', 'wallpaper samples']
            const c_rain =['the rain', 'the centuries', 'our children', 'the greatest
minds', 'some slugs', "our handiwork", 'a primadonna', 'some sounds']
            const c_flow =['flow into', 'trip over', 'berate', 'empty', 'ripple',
'line', 'swell', 'make homes of', 'brew sodas in', 'see through', 'get lost in']
            const c_swans =['swans', 'leaves', 'dreams', 'tricksters', 'grooms',
'winds', 'willows', 'large lakes', 'new doors']
            const c_rocked = ['rocked side to side', 'looked for the right fit',
'wrestled with fate', 'chugged down rivers', 'reached for the sky', 'did their close
readings', 'tried to remember']
            const c_were =['were', 'traveled far', 'filled out', 'reached out', 'grew
to be', 'stole', 'lasted long', 'shot up tall', 'autofilled']
            const c_hold =['hold', 'get past', 'bind', 'grow', 'cushion', 'charm',
'scissor through', 'blur', 'curl', 'climb']

            function flip () {
                document.getElementById("nouna").innerHTML =
varnouna[Math.floor(Math.random() * varnouna.length)];
```

```
                document.getElementById("nounb").innerHTML =
varnounb[Math.floor(Math.random() * varnounb.length)];
                document.getElementById("verba").innerHTML =
varverba[Math.floor(Math.random() * varverba.length)];
                document.getElementById("verbb").innerHTML =
varverbb[Math.floor(Math.random() * varverbb.length)];
                document.getElementById("adja").innerHTML =
varadja[Math.floor(Math.random() * varadja.length)];
                document.getElementById("verbc").innerHTML =
varverbc[Math.floor(Math.random() * varverbc.length)];
                document.getElementById("adjb").innerHTML =
varadjb[Math.floor(Math.random() * varadjb.length)];
                document.getElementById("verbd").innerHTML =
varverbd[Math.floor(Math.random() * varverbd.length)];
                document.getElementById("nounc").innerHTML =
varnounc[Math.floor(Math.random() * varnounc.length)];
                document.getElementById("verbe").innerHTML =
varverbe[Math.floor(Math.random() * varverbe.length)];
                document.getElementById("nound").innerHTML =
varnound[Math.floor(Math.random() * varnound.length)];
                document.getElementById('grand').innerHTML =
c_grand[Math.floor(Math.random() * c_grand.length)];

                document.getElementById('bird').innerHTML =
c_bird[Math.floor(Math.random() * c_bird.length)];
                document.getElementById('go').innerHTML = c_go[Math.floor(Math.random()
* c_go.length)];
                document.getElementById('riled').innerHTML =
c_riled[Math.floor(Math.random() * c_riled.length)];
                document.getElementById('mutters').innerHTML =
c_mutters[Math.floor(Math.random() * c_mutters.length)];
                document.getElementById('goats').innerHTML =
c_goats[Math.floor(Math.random() * c_goats.length)];
                document.getElementById('rush').innerHTML =
c_rush[Math.floor(Math.random() * c_rush.length)];
                document.getElementById('good').innerHTML =
c_good[Math.floor(Math.random() * c_good.length)];
                document.getElementById('rip').innerHTML =
c_rip[Math.floor(Math.random() * c_rip.length)];
                document.getElementById('hearts').innerHTML =
c_hearts[Math.floor(Math.random() * c_hearts.length)];
```

```
                document.getElementById('horizon').innerHTML =
c_horizon[Math.floor(Math.random() * c_horizon.length)];

                document.getElementById('train').innerHTML =
c_train[Math.floor(Math.random() * c_train.length)];
                document.getElementById('shorts').innerHTML =
c_shorts[Math.floor(Math.random() * c_shorts.length)];
                document.getElementById('filed').innerHTML =
c_filed[Math.floor(Math.random() * c_filed.length)];
                document.getElementById('past').innerHTML =
c_past[Math.floor(Math.random() * c_past.length)];
                document.getElementById('feel').innerHTML =
c_feel[Math.floor(Math.random() * c_feel.length)];
                document.getElementById('trees').innerHTML =
c_trees[Math.floor(Math.random() * c_trees.length)];
                document.getElementById('rain').innerHTML =
c_rain[Math.floor(Math.random() * c_rain.length)];
                document.getElementById('flow').innerHTML =
c_flow[Math.floor(Math.random() * c_flow.length)];
                document.getElementById('swans').innerHTML =
c_swans[Math.floor(Math.random() * c_swans.length)];
                document.getElementById('rocked').innerHTML =
c_rocked[Math.floor(Math.random() * c_rocked.length)];
                document.getElementById('were').innerHTML =
c_were[Math.floor(Math.random() * c_were.length)];
                document.getElementById('hold').innerHTML =
c_hold[Math.floor(Math.random() * c_hold.length)];
                }
        </script>

        <script
src="https://cdnjs.cloudflare.com/ajax/libs/html2canvas/1.3.4/html2canvas.min.js"></sc
ript>

        <script
src='https://ajax.googleapis.com/ajax/libs/jquery/3.6.4/jquery.min.js'></script>

        <script>
            function ssa () {
            html2canvas(document.getElementById("wholea")).then((canvas) => {
                let a = document.createElement("a");
                a.download = "ss-a.png";
```

```
            a.href = canvas.toDataURL("image/png");
            a.click();
        });}

        function ssb () {
        html2canvas(document.getElementById("wholeb")).then((canvas) => {
            let a = document.createElement("a");
            a.download = "ss-b.png";
            a.href = canvas.toDataURL("image/png");
            a.click();
        });}

        function ssc () {
        html2canvas(document.getElementById("wholec")).then((canvas) => {
            let a = document.createElement("a");
            a.download = "ss-c.png";
            a.href = canvas.toDataURL("image/png");
            a.click();
        });}

        $(".back").click(function(){
        $(".borders, #corner").toggle();
        });

        const isComp = window.matchMedia('(min-width: 600px)')

        if (isComp.matches) {

        $(document).on("click mousemove",function(e){
        var x = e.clientX;
        var y = e.clientY;
        var newposX = y/2 + x/2;
        var newposY = x/2 ;
$("#wholea").css("transform","translate3d("+newposX+"px,"+newposY+"px,0px)");
        });

        $(document).on("click mousemove",function(e){
        var x = e.clientX;
        var y = e.clientY;
        var newposX = y/4 + x/4;
        var newposY = x/4;
$("#wholec").css("transform","translate3d("+newposX+"px,"+newposY+"px,0px)");
```

```
            });

            $(document).on("click mousemove",function(e){
            var x = e.clientX;
            var y = e.clientY;
            var newposX = y/6 + y/6;
            var newposY = x/6;
$("#wholeb").css("transform","translate3d("+newposX+"px,"+newposY+"px,0px)");
            });}
        </script>
    </body>
</htmL>
```